

Step 1 — A Script That Sends One Message and Prints the Reply — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real, unattended service would check this). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The pieces, and where each one lives](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The pieces, and where each one lives

Basic Version

This step only proves one thing: your code can reach the model and get a sensible reply back. Nothing else yet — no loop, no memory, no streaming.

You need three small pieces: a config loader that reads your API key (reuse Doc01's, don't rewrite it), an OpenAI client built from that key, and one function that sends a single message and hands back the reply text.

Things to use: - `from openai import OpenAI` — the client class. - `client = OpenAI(api_key=...)` — build it once. - `client.chat.completions.create(model=..., messages=[...])` — the actual call. - Each message is a dict: `{"role": "user", "content": "your text"}` . - The reply text is at `response.choices[0].message.content` .

Don't overthink the message format yet — one message in, one string reply out.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The real shape you're aiming for is two small functions in `chat_client.py` :

```
def create_client(config: Config) -> OpenAI:
```

and

```
def send_message(client: OpenAI, message: str) -> str:
```

`create_client()` should read `config.openai_api_key` (from Doc01's `load_config()`) and return an `OpenAI(api_key=...)` instance — don't let the SDK silently fall back to reading `OPENAI_API_KEY` from the environment on its own; be explicit, so a missing key fails at `load_config()` with your own clear error, not deep inside the SDK.

`send_message()` should call `client.chat.completions.create(model=..., messages=[{"role": "user", "content": message}])` and return `response.choices[0].message.content`. Even for one message, `messages` is still a list of dicts, because that's the shape the API always expects — Step 2 will just make this list grow. `response.choices[0]` — the API always returns a list of choices even when you only asked for one; `[0]` is the first (and, by default, only) one.

Sketch both signatures, and where each piece of data (the key, the message, the reply) flows, before writing the bodies.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

`send_message()` proves the connection works, but only by accident — it happens to test whatever message you first send it. If the user's very first real message is also the first time your code touches the API, a bad key shows up as a confusing crash mixed into the app's first real interaction, not as a clear, separate failure.

A real project checks the key on purpose, once, before the user types anything — a cheap test call right after `create_client()`. Two honest ways to do that cheaply:

- `client.models.list()` — lists the models your key can see. Costs no completion tokens at all, and fails immediately with `AuthenticationError` if the key is wrong.
- A 1-token chat completion (`max_tokens=1`) — costs a sliver of a token, but proves the *exact* call path your app actually depends on (the completions endpoint itself), not just that the key authenticates in general.

Now think about a bigger version of this. Imagine this script grows into a real running service — a support desk API that other systems call into, not just a terminal app you run yourself. A real health-check for a service like that usually asks two separate questions, on two different schedules:

- **Liveness** — "is the process even running at all?" Checked every few seconds, so the service can be restarted if it hangs.
- **Readiness** — "can this process actually do its job right now?" Checked less often. This is where an API-key check belongs — it asks whether the thing the service depends on (your OpenAI key, here) is currently working.

Your terminal app's startup key check is really the readiness half of that pattern. You just check it once, at boot, and trust it for the rest of the session, instead of checking it again and again on a schedule. Don't run this check on every single message the user sends, though — that would double your API cost and slow down every turn, for a check that almost never changes mid-session.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate prove the connection works, but only by accident, the first time `send_message()` happens to run. Advanced adds a separate check that runs on purpose, before anything else, so a bad key fails fast with one clear sentence. It also ties that habit to the readiness/liveness split real backend services use — so "check the key at startup" stops being an arbitrary rule, and becomes one example of a pattern you'll see again.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
load config (gets the API key, fails loudly if missing)

make a client using that key

define send_message(client, message):
    call the model with one message
    return just the reply text

in main.py:
    build the client
    call send_message with "What's 2+2?"
    print the reply
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The same plan, closer to real structure:

```
config.py:
    load_config() -> Config          (from Doc01, reused as-is)

chat_client.py:
    def create_client(config: Config) -> OpenAI:
        return OpenAI(api_key=config.openai_api_key)

    def send_message(client: OpenAI, message: str) -> str:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": message}],
        )
        return response.choices[0].message.content

main.py:
    config = load_config()
    logger = get_logger(__name__)
    client = create_client(config)
    reply = send_message(client, "What's 2+2?")
    print(reply)
```

Notice `main.py` calls `load_config()` and `get_logger()` too, even in this tiny step — that habit (config + logger first, before anything else) is what Doc01's Build Task set you up for, and every later step in this project builds on it.

Write this yourself, run it, and confirm you get a sane reply before looking at Advanced.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the missing piece from Hint 1 — a startup check, wired in before the real call:

```
from openai import OpenAI, AuthenticationError

def create_client(config: Config) -> OpenAI:
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client: OpenAI) -> None:
    try:
        client.models.list()
    except AuthenticationError:
        raise SystemExit("Your OPENAI_API_KEY is invalid. Fix your .env before co
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, send_message, check_api_key_at_startup

config = load_config()
logger = get_logger(__name__)
client = create_client(config)
check_api_key_at_startup(client)

reply = send_message(client, "What's 2+2?")
print(reply)
```

Try this with a deliberately wrong key in your `.env` (a fake string, not your real one) and confirm you get the clear `SystemExit` message *before* `send_message()` ever runs — not an SDK traceback buried underneath your test message.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate only find out the key is bad when `send_message()` itself fails — wherever that happens to sit in the code. Advanced adds one dedicated `check_api_key_at_startup()` step, called right after the client is built and before anything else runs. Step 3 will build on this exact shape once there's more than one thing that can go wrong.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)